

|                            |                                                                                                                                                   |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Name of the Student</b> | <b>V Sunil Kumar</b>                                                                                                                              |
| <b>Reg. No</b>             | <b>192425292</b>                                                                                                                                  |
| <b>GitHub Link</b>         | <a href="https://github.com/192425292simats-byte/MLA0403-DEEP-LEARNING.git">https://github.com/192425292simats-byte/MLA0403-DEEP-LEARNING.git</a> |

## SIMATS ENGINEERING

### CO5 - ASSESSMENT TOOL 1

#### Design Desk

**COURSE NAME: Deep Learning**  
**SLOT : A**

**COURSE CODE: MLA04**  
**Faculty : Dr.Arul Raj A.M**

---

#### **COURSE OUTCOME COVERED**

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

**Course Outcome Weightage: 35%**  
**Assessment Tool Weightage: 30%**

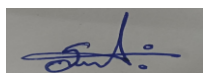
**Duration: 90 minutes**  
**Mark : 25**

---

#### **Code of Conduct:**

I D.Philip/192425348 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



**Signature of the student:**

**Name of the student: V Sunil**

---

| <b>Criteria</b>  | <b>Subject Knowledge and Conceptual Understanding (2)</b> | <b>Technical Accuracy and Application of Concepts (3)</b> | <b>Problem-Solving and Analytical Thinking (2)</b> | <b>Communication Skills and Clarity of Explanation (2)</b> | <b>Confidence, Presentation and Professional Behaviour (1)</b> | <b>Total (10)</b> |
|------------------|-----------------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------|-------------------|
| <b>Weightage</b> | <b>20 %</b>                                               | <b>30%</b>                                                | <b>20%</b>                                         | <b>20 %</b>                                                | <b>10 %</b>                                                    | <b>100%</b>       |
| <b>Q1</b>        |                                                           |                                                           |                                                    |                                                            |                                                                |                   |
| <b>Q2</b>        |                                                           |                                                           |                                                    |                                                            |                                                                |                   |

|       |  |  |  |  |  |  |
|-------|--|--|--|--|--|--|
| Q3    |  |  |  |  |  |  |
| Q4    |  |  |  |  |  |  |
| Q5    |  |  |  |  |  |  |
| Total |  |  |  |  |  |  |

**Faculty Signature :**

**Total Marks:**

---

**QUESTIONS: 25**

**Total Marks:25**

### **1. Design an Autoencoder-Based Feature Learning System**

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

### **2. Design a Robust Regularized Autoencoder**

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

### **3. Design a Deep Generative Model**

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

### **4. Design a Stochastic and Contractive Representation Model**

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

### **5. Design a GAN-Based Data Generation System**

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow, adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

## **Design Desk Guidelines**

- Identify the real-world problem, requirements, inputs, outputs and constraints.
- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.
- Mention suitable evaluation measures and expected outcomes.
- Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

ANSWERS:

# 1. Design an Autoencoder-Based Feature Learning System

## Problem

A manufacturing company collects high-dimensional sensor data from machines and products. The data may contain hundreds of features such as:

- Temperature
- Pressure
- Vibration
- Speed
- Voltage
- Current
- Acoustic signals
- Manufacturing measurements

Using all features directly is computationally expensive and may contain redundant information.

The objective is to design an **Undercomplete Autoencoder** that learns a compact representation of the sensor data and helps identify important features for defect detection.

## Requirements

The system should:

1. Accept high-dimensional sensor data.
2. Reduce the number of dimensions.
3. Learn important hidden features automatically.
4. Reconstruct the original input.
5. Preserve useful information.
6. Provide compressed features for a downstream defect-classification system.

## Input and Output

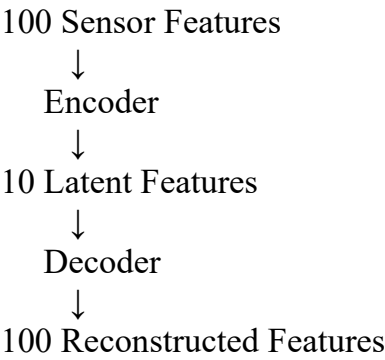
**Input:**

$X=[x_1, x_2, \dots, x_n]$

For example, suppose each product has **100 sensor features**.

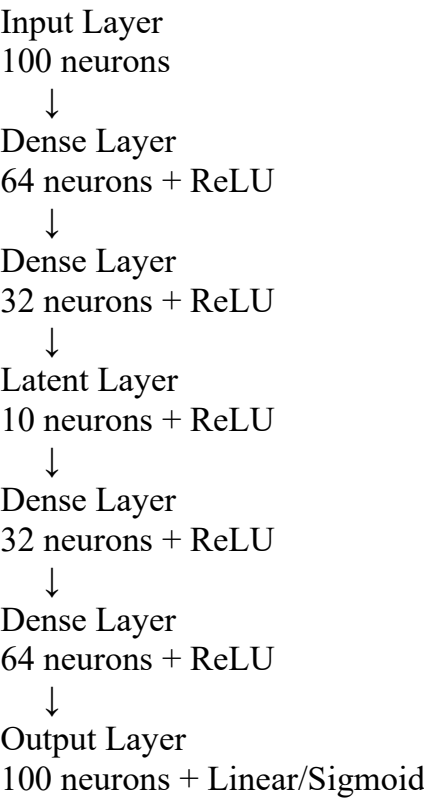
**Output:**

A compressed latent representation containing, for example, **10 features**.



## Architecture

A suitable architecture is:



### Architecture Table

| Layer     | Units | Activation | Purpose          |
|-----------|-------|------------|------------------|
| Input     | 100   | —          | Sensor features  |
| Encoder 1 | 64    | ReLU       | Extract features |

|           |            |                        |
|-----------|------------|------------------------|
| Encoder 2 | 32 ReLU    | Further compression    |
| Latent    | 10 ReLU    | Compact representation |
| Decoder 1 | 32 ReLU    | Expand representation  |
| Decoder 2 | 64 ReLU    | Recover information    |
| Output    | 100 Linear | Reconstruct input      |

## Encoder

The encoder converts the original high-dimensional input into a smaller representation.

$$z = f_{\theta}(x)$$

where:

- $x$  = original sensor data
- $z$  = latent representation
- $f_{\theta}$  = encoder

For example:

$$100 \rightarrow 64 \rightarrow 32 \rightarrow 10$$

So 100 original features are represented using only 10 latent features.

## Latent Representation

The **latent layer** is the most important part of the undercomplete autoencoder.

Example:

Original:

[100 sensor features]

Latent:

[10 learned features]

These 10 features may represent hidden characteristics such as:

- Machine vibration pattern
- Temperature behavior
- Pressure pattern
- Mechanical stress
- Product quality characteristics

The system learns these features automatically rather than manually selecting them.

## Decoder

The decoder reconstructs the original input:

$\hat{x} = g\phi(z)$   
where:

- $z$  = latent representation
- $\hat{x}$  = reconstructed input

The decoder performs:

$10 \rightarrow 32 \rightarrow 64 \rightarrow 100$

## Activation Functions

### ReLU

ReLU is suitable for hidden layers:

$\text{ReLU}(x) = \max(0, x)$

Advantages:

- Simple
- Fast
- Helps deep networks learn nonlinear patterns
- Reduces vanishing-gradient problems

### Output Activation

For normalized continuous sensor values, **linear activation** can be used.

For values normalized between 0 and 1, **sigmoid** can be used.

## Reconstruction Loss

The autoencoder tries to make the reconstructed data as close as possible to the original data.

Mean Squared Error is suitable:

$L = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$

The objective is:

$\min L(x, \hat{x})$

Lower reconstruction error means better reconstruction.

# How Dimensionality Reduction Happens

Suppose:

Original data = 100 features



Encoder



Latent representation = 10 features

Therefore:

100→10

This is a **90% reduction in dimensionality**.

The model cannot simply copy the input because the latent layer has fewer neurons than the input.

Therefore, it must learn the most important information.

## Training Process

1. Collect sensor data.
2. Normalize the data.
3. Send data to encoder.
4. Encoder generates latent representation.
5. Decoder reconstructs the input.
6. Calculate reconstruction error.
7. Backpropagate the error.
8. Update network weights.
9. Repeat for multiple epochs.

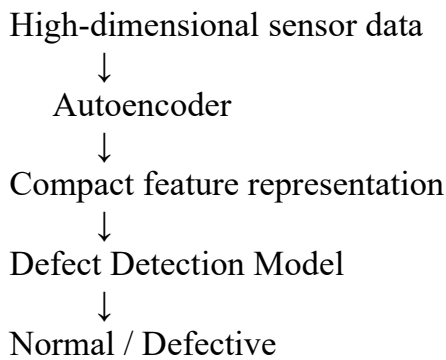
## Evaluation Measures

Suitable measures include:

- Mean Squared Error
- Mean Absolute Error
- Reconstruction error
- Explained variance
- Classification accuracy using latent features
- Precision
- Recall
- F1-score

## Expected Outcome

The system produces:



### Advantages

- Reduces dimensionality.
- Removes redundant information.
- Learns features automatically.
- Reduces computational requirements.
- Useful for downstream classification.

### Limitation

If the latent dimension is too small, important information may be lost.

## 2. Design a Robust Regularized Autoencoder

### Problem

Healthcare data often contains:

- Missing values
- Measurement errors
- Noise
- Redundant features
- Irrelevant features

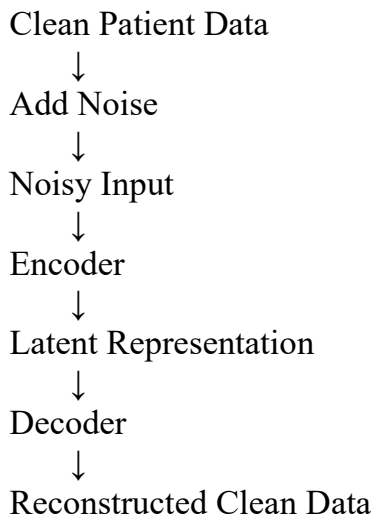
A normal autoencoder may memorize the training data.

Therefore, we need a **Regularized Autoencoder** that learns useful and robust representations.

### Proposed Model

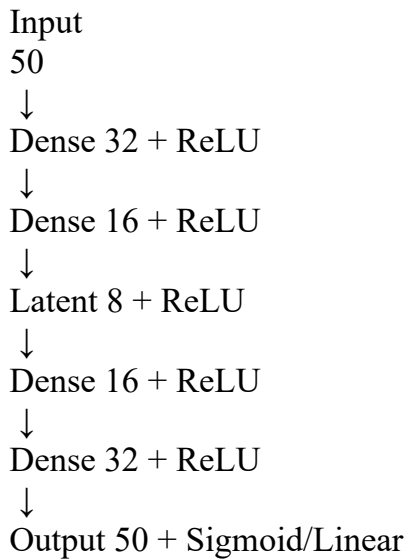
A good choice is a **Denoising Autoencoder with L2 regularization**.

The model intentionally receives corrupted/noisy input but tries to reconstruct the original clean input.



## Example Architecture

Suppose there are 50 patient-record features.



## Architecture Table

| Layer   | Units | Activation |
|---------|-------|------------|
| Input   | 50    | —          |
| Encoder | 32    | ReLU       |

|         |                   |
|---------|-------------------|
| Encoder | 16 ReLU           |
| Latent  | 8 ReLU            |
| Decoder | 16 ReLU           |
| Decoder | 32 ReLU           |
| Output  | 50 Linear/Sigmoid |

## Regularization Mechanism

### 1. Noise Injection

Random noise is added to the input:

$$\tilde{x} = x + \epsilon$$

where:

$$\epsilon \sim N(0, \sigma^2)$$

The network receives noisy data but tries to reconstruct the clean data.

This prevents the model from simply memorizing the training examples.

### 2. L2 Regularization

An L2 penalty can be added to the loss:

$$L = L_{\text{reconstruction}} + \lambda \sum w^2$$

where:

- $\lambda$  = regularization strength
- $w$  = model weights

Large weights are penalized.

## Complete Training Objective

$$L = n \frac{1}{2} \sum (x - \hat{x})^2 + \lambda \sum w^2$$

The model minimizes both:

1. Reconstruction error
2. Weight complexity

## Why Regularization Helps

Without regularization:

Training data  
↓  
Autoencoder  
↓  
May memorize data

With regularization:

Noisy/variable data  
↓  
Regularized Autoencoder  
↓  
Robust latent representation  
↓  
Better generalization

## Training Process

1. Collect patient features.
2. Clean and normalize the data.
3. Add controlled noise.
4. Send corrupted input to encoder.
5. Generate latent representation.
6. Decode the representation.
7. Compare reconstruction with original clean input.
8. Add L2 penalty.
9. Calculate total loss.
10. Backpropagate.
11. Update weights.

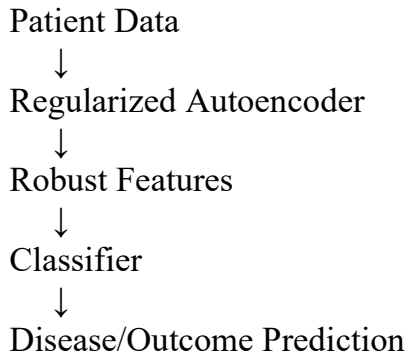
## Evaluation

The model can be evaluated using:

- Reconstruction MSE
- MAE
- Classification accuracy
- Precision
- Recall
- F1-score
- AUC
- Performance on noisy test data

## Expected Outcome

The learned latent features can be given to a classifier:



## Advantages

- Reduces overfitting.
- Handles noisy data.
- Learns robust representations.
- Improves generalization.
- Useful for downstream classification.

## Limitation

Too much regularization may remove useful information.

# 3. Design a Deep Generative Model

## Problem

A recommendation/content-generation system needs to learn complex patterns in data.

For this application, we can design a **Deep Belief Network (DBN)**.

A DBN consists of multiple layers of hidden variables that learn hierarchical representations.

## Proposed Application

### Movie Recommendation System

Input information can include:

- User preferences
- Movie ratings

- Genre preferences
- Watch history
- User behavior

The model learns hidden patterns between users and movies.

## Architecture

Example:

Visible Layer

100 units

(User/movie features)



Hidden Layer 1

64 units



Hidden Layer 2

32 units



Hidden Layer 3

16 units



Output/Recommendation

10 units

A DBN can be constructed using stacked Restricted Boltzmann Machines (RBMs).

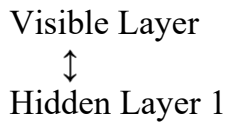
## Architecture Table

| Layer    | Units | Purpose                      |
|----------|-------|------------------------------|
| Visible  | 100   | Input user/movie information |
| Hidden 1 | 64    | Low-level patterns           |
| Hidden 2 | 32    | Higher-level patterns        |
| Hidden 3 | 16    | Complex representations      |
| Output   | 10    | Recommendation scores        |

# Learning Process

DBNs commonly use **layer-wise unsupervised learning**.

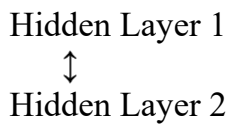
## Step 1: Train First RBM



The first layer learns basic patterns.

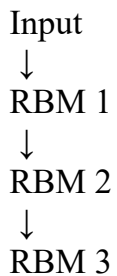
## Step 2

The learned hidden representation becomes input for the next RBM:



## Step 3

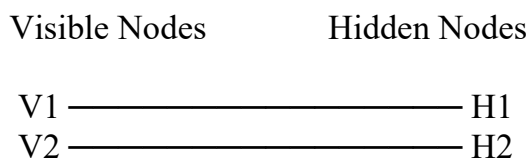
Continue for deeper layers.

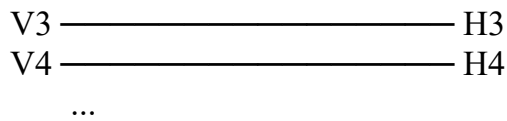


Finally, the entire network can be fine-tuned using supervised learning.

# RBM Structure

An RBM contains:





There are connections between visible and hidden layers but no connections within the same layer.

## Learning Mechanism

RBM training uses energy-based learning.

The energy function can be represented as:

$$E(v,h) = -i \sum a_i v_i - j \sum b_j h_j - i,j \sum v_i w_{ij} h_j$$

The model learns weights that represent the probability distribution of the training data.

Methods such as **Contrastive Divergence** can be used to train RBMs.

## Why DBN?

A DBN is useful because it can:

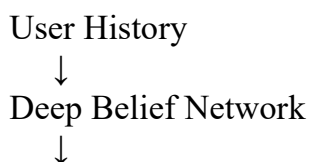
- Learn hierarchical features.
- Model complex probability distributions.
- Perform unsupervised feature learning.
- Work with high-dimensional data.
- Discover hidden relationships.

## Evaluation

For recommendation systems:

- Precision@K
- Recall@K
- NDCG
- RMSE
- MAE
- Recommendation diversity
- User satisfaction

## Expected Outcome



Hidden Preference Representation



Recommendation Scores



Top-N Recommendations

### Advantages

- Learns complex hidden relationships.
- Supports unsupervised pretraining.
- Useful for feature learning.
- Can model complicated data distributions.

### Limitations

- Training is more complex than a simple autoencoder.
- Computationally expensive.
- Modern deep generative models such as VAEs and diffusion models may be preferred for many generation tasks.

## 4. Design a Stochastic and Contractive Representation Model

### Problem

Consider an image-processing application such as **face recognition**.

The same face may appear with:

- Slight rotation
- Small translation
- Different lighting
- Small noise
- Minor expression changes

We need a representation that remains stable under these small changes.

A suitable approach is a **Contractive Autoencoder (CAE)**.

### Objective

The model should learn:

Similar inputs → Similar latent representations.

For example:

Original Face  
↓  
Encoder  
↓  
Latent Representation A

Slightly Modified Face  
↓  
Encoder  
↓  
Latent Representation  $\approx A$

## Architecture

Input Image  
 $64 \times 64 \times 3$   
↓  
Convolution Layer  
32 filters  
↓  
Max Pooling  
↓  
Convolution Layer  
64 filters  
↓  
Max Pooling  
↓  
Latent Representation  
128 features  
↓  
Decoder  
↓  
Reconstructed Image  
 $64 \times 64 \times 3$

## Contractive Mechanism

A normal autoencoder minimizes reconstruction error.

A contractive autoencoder adds a penalty that forces the encoder to be less sensitive to small changes in input.

The loss is:

$$L = L_{\text{reconstruction}} + \lambda ||J_f(x)||_F^2$$

where:

- $J_f(x)$  = Jacobian matrix of encoder
- $\lambda$  = regularization coefficient
- $||J_f(x)||_F^2$  = squared Frobenius norm

## Meaning of the Jacobian Penalty

The Jacobian describes how much the latent representation changes when the input changes.

Without the contractive penalty:

Small input change



Large feature change

With the penalty:

Small input change



Small feature change

Therefore, the representation becomes more stable.

## Activation Functions

For convolutional hidden layers:

**ReLU**

$$\text{ReLU}(x) = \max(0, x)$$

For output images:

- Sigmoid for normalized pixel values between 0 and 1.
- Linear can be used depending on preprocessing.

## Training Process

1. Input image is provided.
2. Encoder extracts features.
3. Latent representation is generated.
4. Decoder reconstructs image.

5. Reconstruction loss is calculated.
6. Contractive penalty is calculated.
7. Both losses are combined.
8. Backpropagation updates weights.

## Evaluation

Suitable metrics:

- Reconstruction MSE
- MAE
- SSIM
- Classification accuracy
- Robustness under noise
- Robustness under small rotations/translations

## Expected Outcome

Image



Contractive Encoder



Stable Latent Features



Face Classifier



Identity Prediction

## Advantages

- Robust to small variations.
- Learns stable representations.
- Reduces sensitivity to noise.
- Useful for image and signal processing.

## Limitation

The contractive penalty increases computational complexity.

# 5. Design a GAN-Based Data Generation System

## Problem

An organization has only a small number of training images.

For example, a manufacturing company may have only 1,000 images of defective products.

A deep-learning classifier usually performs better with more training examples.

Therefore, we can use a **Generative Adversarial Network (GAN)** to generate synthetic images.

## Proposed Application

### Synthetic Defective Product Image Generation

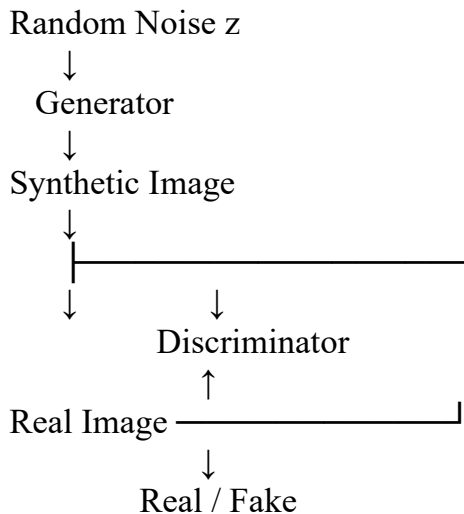
The GAN learns the distribution of real defective-product images and generates new synthetic images.

### GAN Architecture

A GAN contains two neural networks:

1. **Generator**
2. **Discriminator**

Architecture:



### Generator

The Generator creates fake images from random noise.

Input:

$z \sim N(0,1)$

Example:

Random Noise  
100 values  
↓  
Dense 256  
↓  
Dense 512  
↓  
Transpose Convolution  
↓  
Transpose Convolution  
↓  
 $64 \times 64 \times 3$  Image

## Generator Architecture

| Layer          | Units/Filters | Activation |
|----------------|---------------|------------|
| Noise Input    | 100           | —          |
| Dense          | 256           | ReLU       |
| Dense          | 512           | ReLU       |
| Transpose Conv | 256           | ReLU       |
| Transpose Conv | 128           | ReLU       |
| Output         | 3 channels    | Tanh       |

The output image can be normalized to the range:

$[-1,1]$   
when using Tanh.

## Discriminator

The discriminator determines whether an image is real or generated.

Input Image  
 $64 \times 64 \times 3$   
↓

Convolution  
↓  
Leaky ReLU  
↓  
Convolution  
↓  
Leaky ReLU  
↓  
Flatten  
↓  
Dense  
↓  
Sigmoid  
↓  
Real / Fake

## Discriminator Architecture

| Layer   | Filters/Units | Activation |
|---------|---------------|------------|
| Input   | 64×64×3       | —          |
| Conv    | 64            | Leaky ReLU |
| Conv    | 128           | Leaky ReLU |
| Conv    | 256           | Leaky ReLU |
| Flatten | —             | —          |
| Dense   | 1             | Sigmoid    |

## GAN Training Process

GAN training occurs as a competition between two networks.

### Step 1 — Generate Fake Image

Generator receives random noise:

$z \rightarrow \text{Generator} \rightarrow \text{Fake Image}$

## Step 2 — Discriminator Examines Image

The discriminator receives:

Real Image → Discriminator → Real

Fake Image → Discriminator → Fake

## Step 3 — Train Discriminator

The discriminator learns to distinguish real and fake images.

## Step 4 — Train Generator

The generator tries to fool the discriminator.

Generator:

"I want my fake image to be classified as real."

## Step 5

Repeat the process until generated images become realistic.

# GAN Loss Function

The original GAN objective is:

$$\min_D \max_G V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]$$

Where:

- $G$  = Generator
- $D$  = Discriminator
- $x$  = real image
- $z$  = random noise
- $G(z)$  = generated image

## Generator Loss

The generator tries to make the discriminator classify fake images as real.

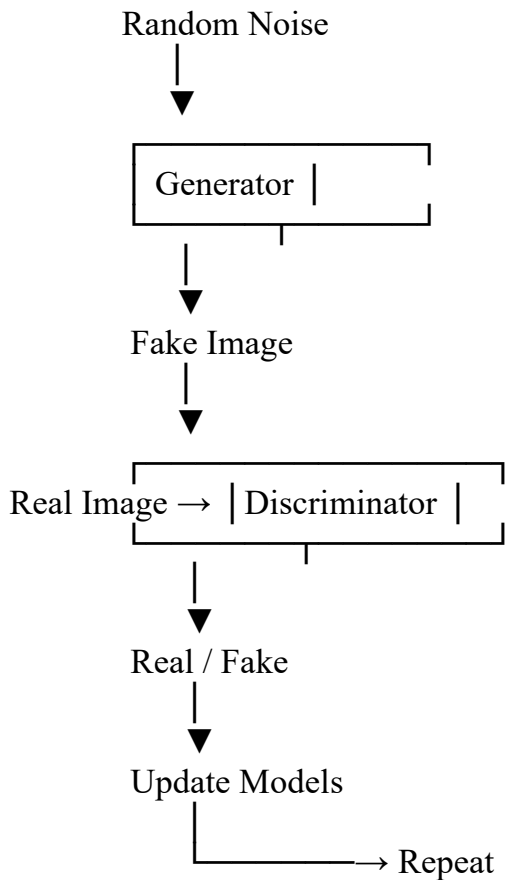
A commonly used non-saturating loss is:

$$L_G = -\mathbb{E}[\log D(G(z))]$$

## Discriminator Loss

$$LD = -E[\log D(x)] - E[\log(1 - D(G(z)))]$$

## Complete GAN Workflow



## Evaluation of GAN

Generating visually realistic images is not enough. We should evaluate both **quality and usefulness**.

### 1. Visual Quality

Human experts inspect generated images.

### 2. FID

**Fréchet Inception Distance (FID)** measures similarity between real and generated image distributions.

Lower FID is generally better.

### 3. IS

**Inception Score (IS)** evaluates image quality and diversity.

### 4. Diversity

Check whether GAN produces many different images rather than repeating the same image.

### 5. Downstream Accuracy

Train a defect classifier using:

Real Images

+

Synthetic Images

Then check whether classification performance improves.

## GAN vs Autoencoder

| Feature                     | Autoencoder                               | GAN                  |
|-----------------------------|-------------------------------------------|----------------------|
| Main purpose                | Representation/reconstruction             | Data generation      |
| Encoder                     | Usually yes                               | No                   |
| Decoder/Generator           | Decoder                                   | Generator            |
| Discriminator               | No                                        | Yes                  |
| Reconstruction              | Excellent                                 | Not necessarily      |
| Image generation quality    | Moderate                                  | Often high           |
| Latent representation       | Important                                 | Random latent vector |
| Training                    | Relatively easier                         | More difficult       |
| Training stability          | Usually stable                            | Can be unstable      |
| Synthetic data generation   | Possible                                  | Main purpose         |
| Suitable for limited images | Less suitable for high-quality generation | Very suitable        |

## Why GAN Is Suitable

The organization needs **new realistic training images**.

An autoencoder mainly learns:

Image  $\rightarrow$  Latent  $\rightarrow$  Reconstructed Image

A GAN learns:

Random Noise  $\rightarrow$  Generator  $\rightarrow$  New Realistic Image

Therefore, GAN is more appropriate when the primary requirement is **synthetic data generation**.

## Comparison of All Five Designs

| Question | Model                             | Main Purpose              | Important Loss                    |
|----------|-----------------------------------|---------------------------|-----------------------------------|
| 1        | Undercomplete Autoencoder         | Dimensionality reduction  | Reconstruction MSE                |
| 2        | Regularized/Denoising Autoencoder | Robust feature learning   | Reconstruction + Regularization   |
| 3        | Deep Belief Network               | Generative modeling       | Energy/probabilistic objective    |
| 4        | Contractive Autoencoder           | Stable representations    | Reconstruction + Jacobian penalty |
| 5        | GAN                               | Synthetic data generation | Adversarial loss                  |

## Easy Way to Remember the Five

### 1. Undercomplete Autoencoder

**Main idea:** Compress

Big Data  $\rightarrow$  Small Latent Features  $\rightarrow$  Reconstruct

Used for **dimensionality reduction**.

### 2. Regularized Autoencoder

**Main idea:** Learn robust features

Noisy Data  $\rightarrow$  Encoder  $\rightarrow$  Robust Features  $\rightarrow$  Decoder

Used when data contains **noise and overfitting problems**.

### 3. Deep Belief Network

## Main idea: Learn deep hidden patterns

Input  $\rightarrow$  Hidden 1  $\rightarrow$  Hidden 2  $\rightarrow$  Hidden 3

Used for **complex probability distributions** and recommendation/generative applications.

## 4. Contractive Autoencoder

## Main idea: Stability

Small Input Change  $\rightarrow$  Small Feature Change

Used for **images/signals** where robustness is important.

## 5. GAN

## Main idea: Generate

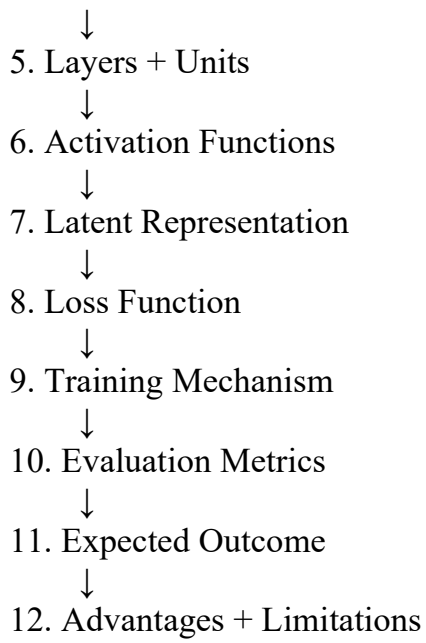
Random Noise  $\rightarrow$  Generator  $\rightarrow$  Fake Image  
 $\uparrow$   
 Discriminator

Used for **synthetic image/data** generation.

## Overall Design Strategy

For a **Design Desk** question, you can follow this sequence for every model:

1. Real-world Problem
- ↓
2. Requirements
- ↓
3. Input
- ↓
4. Architecture



**RUBRICS FOR CALCULATION:**

| <b>Criteria</b>                  | <b>Weightage</b> | <b>Level 4 Exemplary</b>                                                            | <b>Level 3 Proficient</b>                              | <b>Level 2 Developing</b>                           | <b>Level 1 Beginning</b>                                |
|----------------------------------|------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------------|---------------------------------------------------------|
| Understanding of Problem Context | 20               | Clearly understands the problem, requirements, constraints, and expected outcomes.  | Good understanding with minor gaps in requirements.    | Basic understanding; some requirements are unclear. | Poor understanding or misinterpretation of the problem. |
| Application of Concepts          | 30               | Accurately applies appropriate generative deep learning concepts and architectures. | Applies relevant concepts correctly with minor errors. | Applies basic concepts with limited accuracy.       | Fails to apply appropriate concepts.                    |
| Design Approach                  | 20               | Innovative, logical, well-structured, feasible, and technically justified design.   | Logical and feasible design with minor gaps.           | Basic design with some inconsistencies.             | Unclear, illogical, or impractical design.              |
| Accuracy of Solution             | 20               | Completely correct architecture, process, objectives, and justification.            | Mostly correct with minor technical errors.            | Partially correct solution with noticeable gaps.    | Incorrect or incomplete solution.                       |
| Clarity                          | 10               | Clear, well-organized diagram and explanation with proper technical labeling.       | Understandable with minor clarity issues.              | Limited clarity and explanation.                    | Poorly presented and difficult to understand.           |